Chanakya An offline AI teacher: narrated, animated lessons in nine languages

Team name: The Last Commit · **Members:** Aryan Dubey, Priyadarshini Prasad, Niharika K P, Dhiraj Shetty
College: RV University, Bengaluru · **Course:** B.Tech (Hons) · **Branch:** Computer Science Engineering · **Year:** 2nd year
Repo: github.com/signingoff-dubey/UNLOX-TEACHER · **Version:** v0.16.0 · **Built for:** the Unlox hackathon

The problem and our answer

Learners in India need short visual explanations in their own language, but the good ones are online-only, English-first, or built on paid cloud AI. Students on ordinary laptops, often with poor internet, are left out. Chanakya is a local-first teacher: you type a topic (or attach a photo or a PDF) and, with Wi-Fi off, it writes a lesson, narrates it in your language, animates it, and lets you watch it in the browser or export an MP4 with captions. No outside AI service is ever called, and the same product also runs entirely on an Android phone.

How a lesson is made

The one rule that shapes the design is that the AI only returns *data*: a JSON lesson, checked against a schema, that picks one of eleven scene templates and fills in its text. Our own code draws, times and renders every frame, and nothing the model writes is ever executed. The request is first cleaned ("teach me about dsa in java" becomes "DSA in Java"), then the system remembers which related videos the learner already owns, retrieves facts from an offline knowledge base (Simple English Wikipedia, our own reference notes and any PDF the learner attached), and asks the local language model for a plan. The plan is repaired and sent back with feedback if it repeats itself, has thin narration or shows a technical topic only as text; repeated scenes are removed, a visual director swaps weak scene choices, a writer pass lengthens short narration, and a fact checker verifies the statements. The English plan is then translated, each sentence is spoken with a delivery that fits it, and every scene lasts exactly as long as its audio, so voice, captions and visuals never drift.

What it can do

Lessons run from 30 seconds to 4 minutes; a course is one to three videos, and "Auto" lets the AI count the concepts in a topic. The eleven scene types are title, bullets, flow, cycle, compare, diagram, quiz, summary, **graph** (lines, parabolas, circles, ellipses, sine waves and more, with points, tangents and chords), **code** (syntax-coloured, with a highlight that follows the narration) and **network** (boxes and arrows for trees, linked structures and pipelines). Four modes fit the learner: Explanation, Study, Homework (teaches the method and writes the answers into a separate file) and Revise (re-teaches what the learner keeps getting wrong). A photo of a page or a board is read by a local vision model, and PDFs up to 2 GB and 1,000 pages are read as Markdown, with OCR for scans and a description of every picture inside, so a lesson can teach a single chapter with page citations. Around the video there is a tutor chat (text or voice) that can build "one more video" on request, study tools (flashcards, quiz, mind map, glossary, FAQ, timeline), spaced-repetition review, teach-it-back grading, rewards, worksheets and question papers, MP4, captions and audio export, an offline bundle for a USB stick, and Learnverse, a walkable 3D world the local model builds from any topic.

Flash and Pro

Flash shows scenes within seconds and keeps writing while you watch: a short outline, then parts of about 40 seconds written one after another, each told what the earlier parts already taught. **Pro** shows nothing until the whole video is planned: it first outlines every scene, writes the first minute in one call, and patches each further minute on with the whole outline and all earlier text in view, then gives the fact checker twice the time. Both use the same model. On "teach me what avl trees are" (two minutes), Flash finished in 133 seconds with the first scenes watchable after 18, while Pro took 167 seconds and covered balance factor, single and double rotations, insertion, deletion and uses in a more coherent order.

Models used

Everything runs locally, and models served from remote servers are refused. The **language model** for planning, directing, writing, fact checking, the tutor and the study tools is **Qwen2.5-7B-Instruct** (Q4_K_M, Apache-2.0) served by Ollama, fully resident on the 8 GB GPU at about 43 tokens per second with a 6,144-token context. Photos and pictures inside PDFs are read by **LLaVA 7B**. Retrieval uses **nomic-embed-text** for the knowledge base, **multilingual-e5-small** for PDF passages and **bge-reranker-v2-m3** to re-rank them. Translation uses **NLLB-200 distilled 600M** by default, with **AI4Bharat IndicTrans2** (200M, gated) as an option and a back-translation check on every sentence. Narration uses **Piper** voices (English, Hindi, Telugu, Malayalam, Marathi, Bengali), **AI4Bharat Indic-TTS** int8 ONNX voices on the CPU (Tamil, Kannada, Gujarati and others) and **Indic Parler-TTS** on the GPU as a tone-aware fallback. Speech to text is **faster-whisper**; scanned pages go through

Tesseract and then **Unlimited-OCR** when the GPU is idle. A larger model, `qwen3.5:9b`, was benchmarked and rejected: only 72% of it fits on the card and it ran at 17 tokens per second. On the phone the equivalents are **Gemma 4 E2B** (LiteRT-LM), Piper through **sherpa-onnx**, **Whisper** with **Vosk**, and **ML Kit** for translation and OCR.

Languages and voices

Nine languages are supported: English, Hindi, Telugu, Marathi, Bengali, Malayalam, Tamil, Kannada and Gujarati. The lesson is sized for the slowest voice chosen (from about 2.7 words per second in English down to 1.6 in Telugu and Malayalam), a missing voice falls back to the next engine and finally to correctly timed silence, and the Noto fonts bundled with the player draw every script, conjuncts included. If a translation fails the script or meaning check, English is kept instead of showing a bad translation.

Quality and fact checking

Because a 7-billion-parameter model writes fluently but sometimes wrongly, the plan is judged before it is spoken. A **fact checker** works without any outside service: it calculates arithmetic and linear equations, checks a count against its list and a few lists every child learns (the rainbow, the planets in order, the continents), and compares each statement with passages retrieved for it. A verdict only counts if the judge quotes words that really appear in the source and a second, focused question agrees; a correction must pass the same check, otherwise the statement is removed without leaving a scene too thin. A **memory** module tells the planner what the learner already has so videos build on each other instead of repeating hooks and examples, and technical topics are shown with real code (in the language the learner names) or a drawn tree, with at most one everyday comparison, instead of a forced analogy. The report of what was checked, confirmed, corrected or removed is saved with each lesson.

Robustness

Jobs resume after a crash, a GPU out-of-memory falls back to the CPU and retries, a memory gate waits when the laptop is short of RAM or VRAM, calls to Ollama are retried when it is busy, and a readiness check (`/api/ready`) names anything missing and the command that fixes it. The backend has 345 automated tests, and live lessons on a dozen topics were generated in both modes to find and fix the defects the tests could not.

Tech stack

Backend: Python 3.12, FastAPI, Pydantic, SQLite, PyTorch (CUDA 12.8), Transformers, ONNX Runtime, sentence-transformers, FAISS, PyMuPDF and pymupdf4llm. **Web:** React 18, TypeScript, Vite, Remotion for the player and the headless MP4 renderer (Chrome and ffmpeg), three.js for Learnverse, Mermaid for mind maps. **Android:** Kotlin, Jetpack Compose, Room, WorkManager, Media3. **3D lessons:** a headless Blender pipeline. The user interface follows the team's Figma design.

Performance and hardware

Built and measured on an RTX 5060 laptop with 8 GB of VRAM and 15 GB of RAM. The first scene of a short lesson appears about 1.2 seconds after planning starts (6 to 18 seconds from the click in Flash), a two-minute video takes two to three minutes to build, a thousand-page scanned PDF is searchable in under three minutes, and the language model, translator and voices share the card by taking turns. The Android app was tested on a OnePlus 13R.

Known limits

The 7B model still makes occasional factual slips; the checker removes what it can prove wrong and reports the rest as unconfirmed. Code is displayed rather than run (only Python is syntax-checked). The Android app does not yet have Flash and Pro, the fact checker or memory. Wikipedia text is CC BY-SA 4.0 and every lesson lists the articles it used; model and voice licences are listed in the README.

Run it

```
git clone https://github.com/signingoff-dubey/UNLOX-TEACHER.git
.\setup.ps1 -Gpu     # once, online
.\start.ps1          # http://127.0.0.1:8000
```

Needs Ollama (`qwen2.5:7b-instruct-q4_K_M`, `nomic-embed-text`, `llava:7b`). The full documentation, with every version and every bug and its fix, is in `Chanakya-Documentation.pdf`.